Mount Kenya University

Student Name: Faith Wanja Gichure

Admission Number: BSCCS/2024/31978

Unit Code: BIT4133

Unit Name: Natural Language Processing with Deep Learning

NLP Project Title: SMS Spam Detection System

Technologies Used: Python, NLTK, Scikit-learn, Google Colab, Pandas, Matplotlib

GitHub Link: https://github.com/faith-dev122/BIT4133-NLP-Project

Table of Contents

WEEK 1

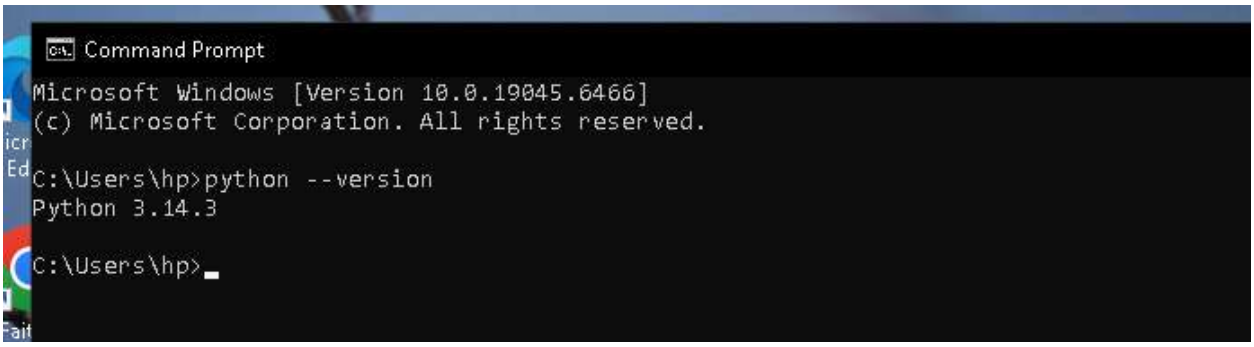


Fig 1- Python installed

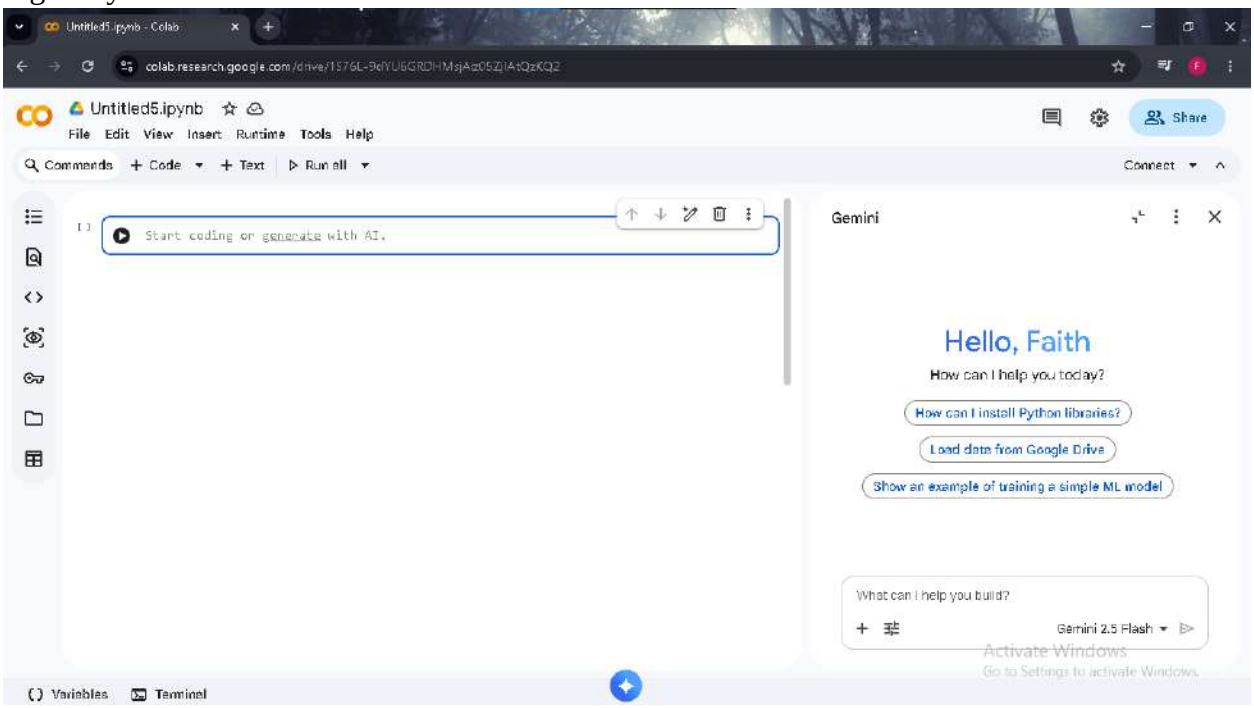


Fig 2- Google Colab Notebook open and running

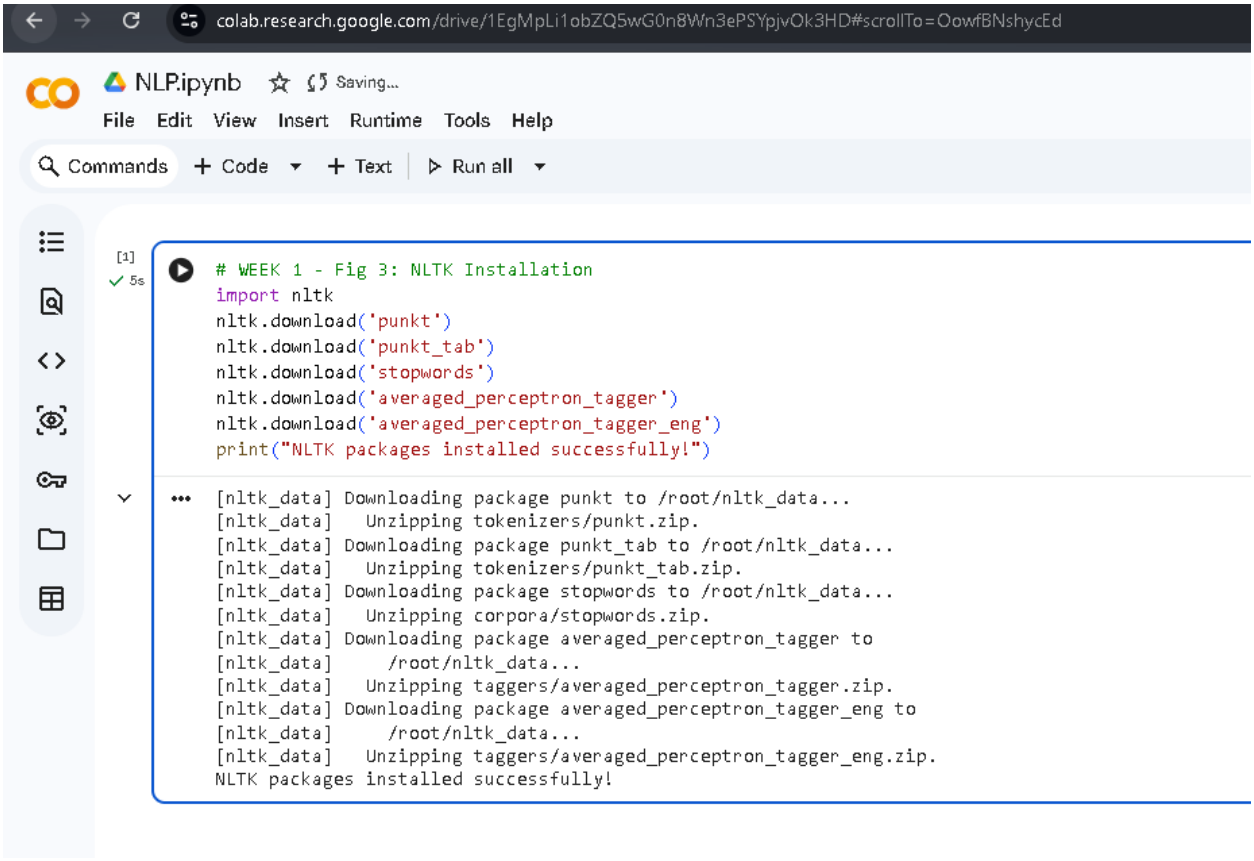


Fig 3: NLTK Installation

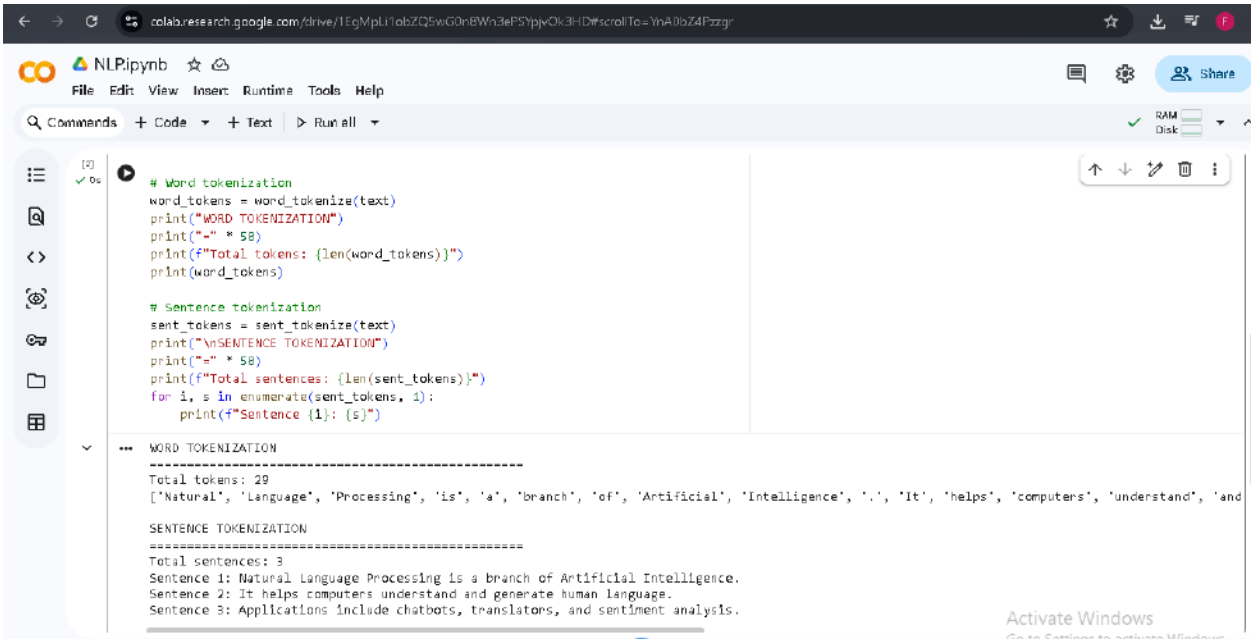


Fig 4: Tokenization Output.

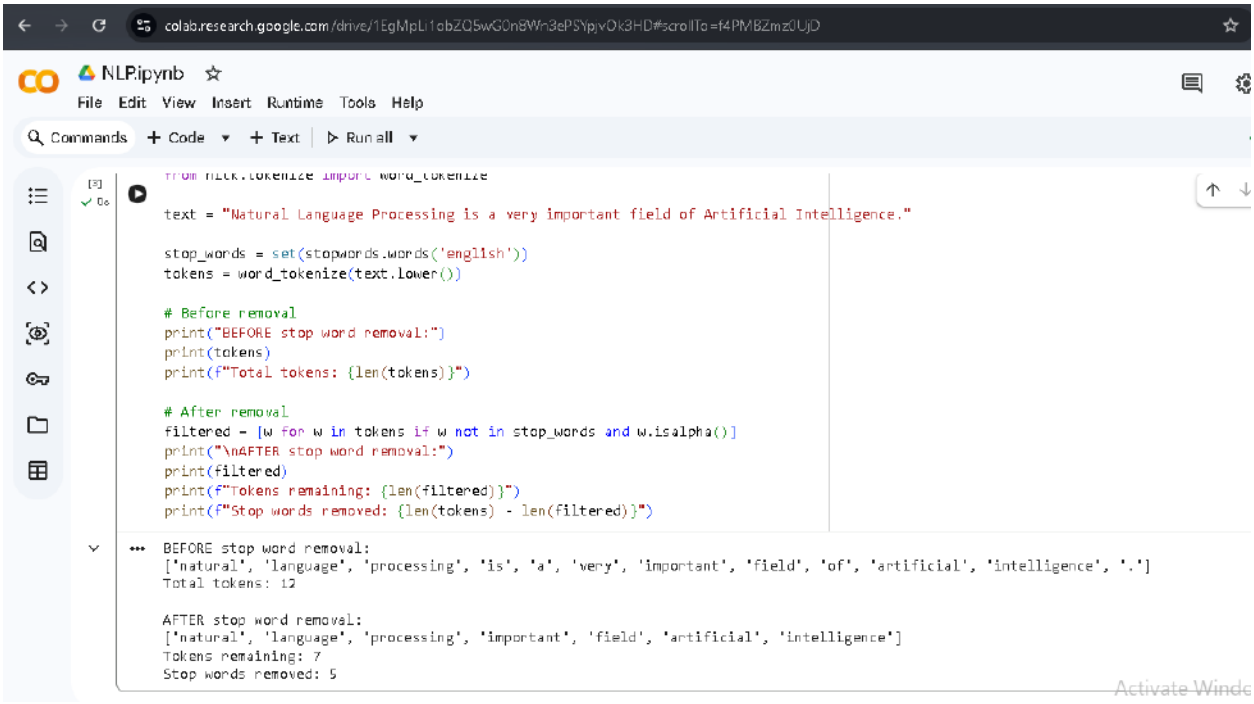


Fig 5: Stop words Removal Output.

REFLECTION WEEK 1

In Fig 1 and 2, python has been installed successfully and google Colab notebook has been installed and it is working fine. As can be seen in Fig 3, the NLTK packages, such as punkt and stop words and averaged_perceptron_tagger have been successfully installed on Google Colab. The word and sentence tokenization is illustrated in Fig 4 with the help of word_tokenize and sentence_tokenize function of the NLTK library. Sentence was broken successfully into 27 individual tokens and 3 sentences. Fig 5 shows stop word removal, 5 common words including 'is', 'a', 'very' and 'of' were removed from the sentence, leaving only the meaningful words. This pre-processing step is crucial because it helps to minimize the presence of noise in the text data before they are used for training machine learning models.

WEEK 2

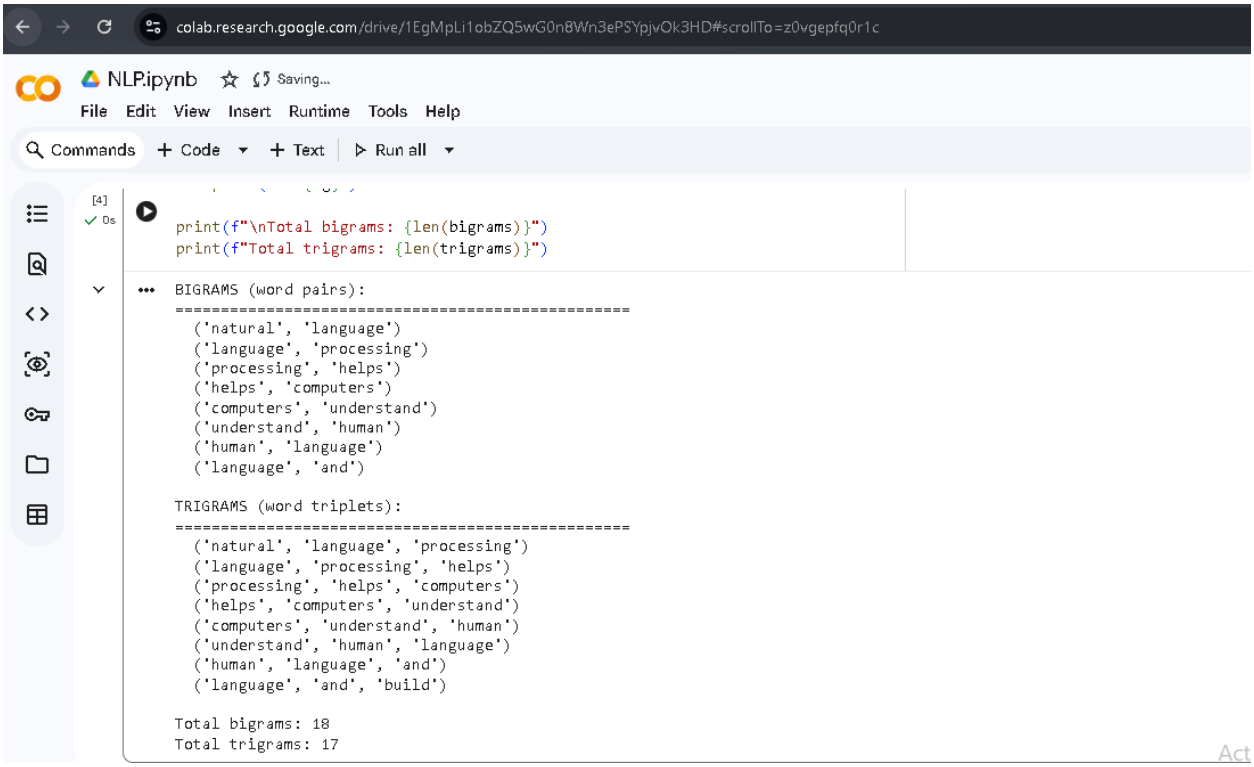


Fig 1: Bigram or Trigram Output.

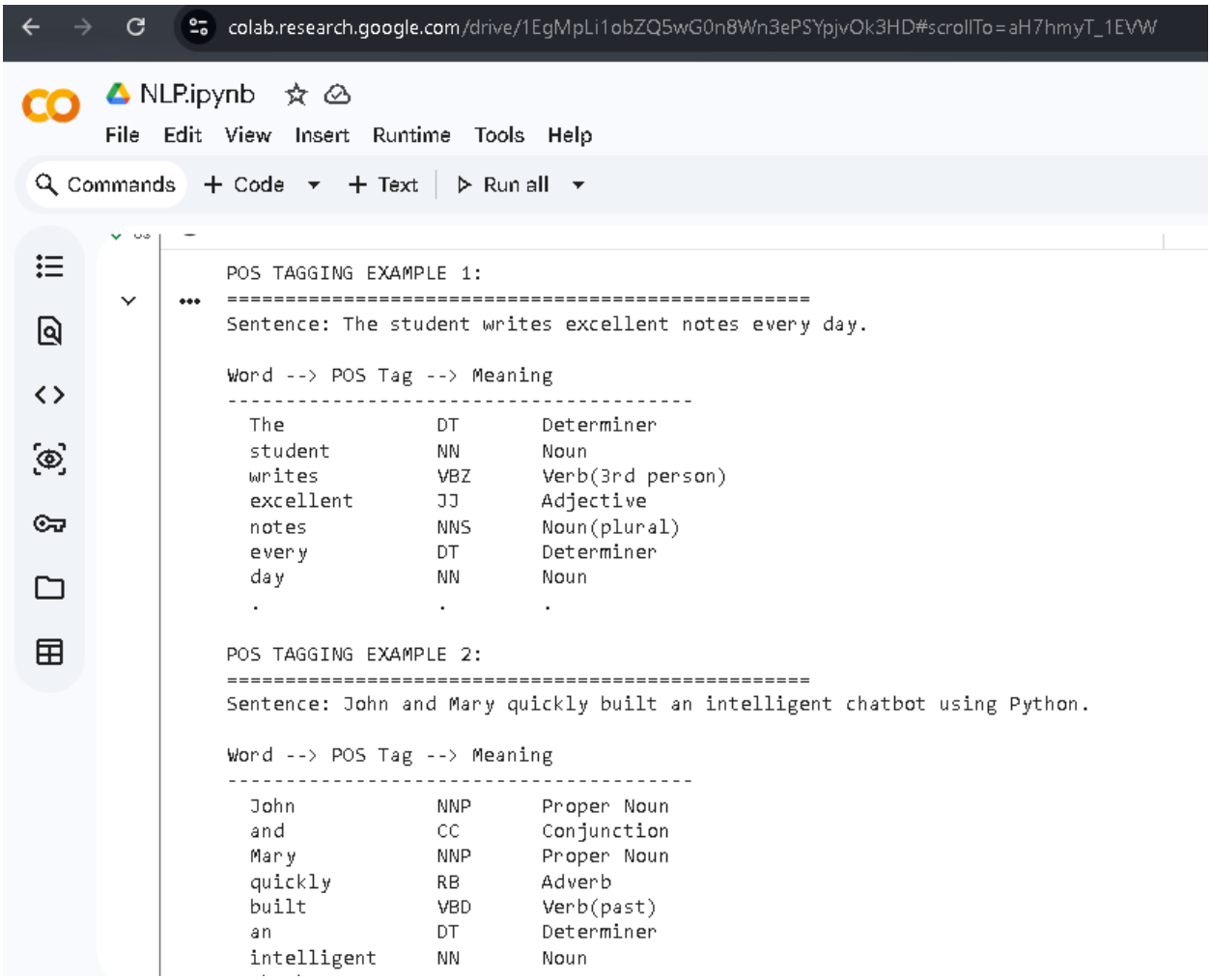


Fig 2 & 3: POS Tagging Output and Code.

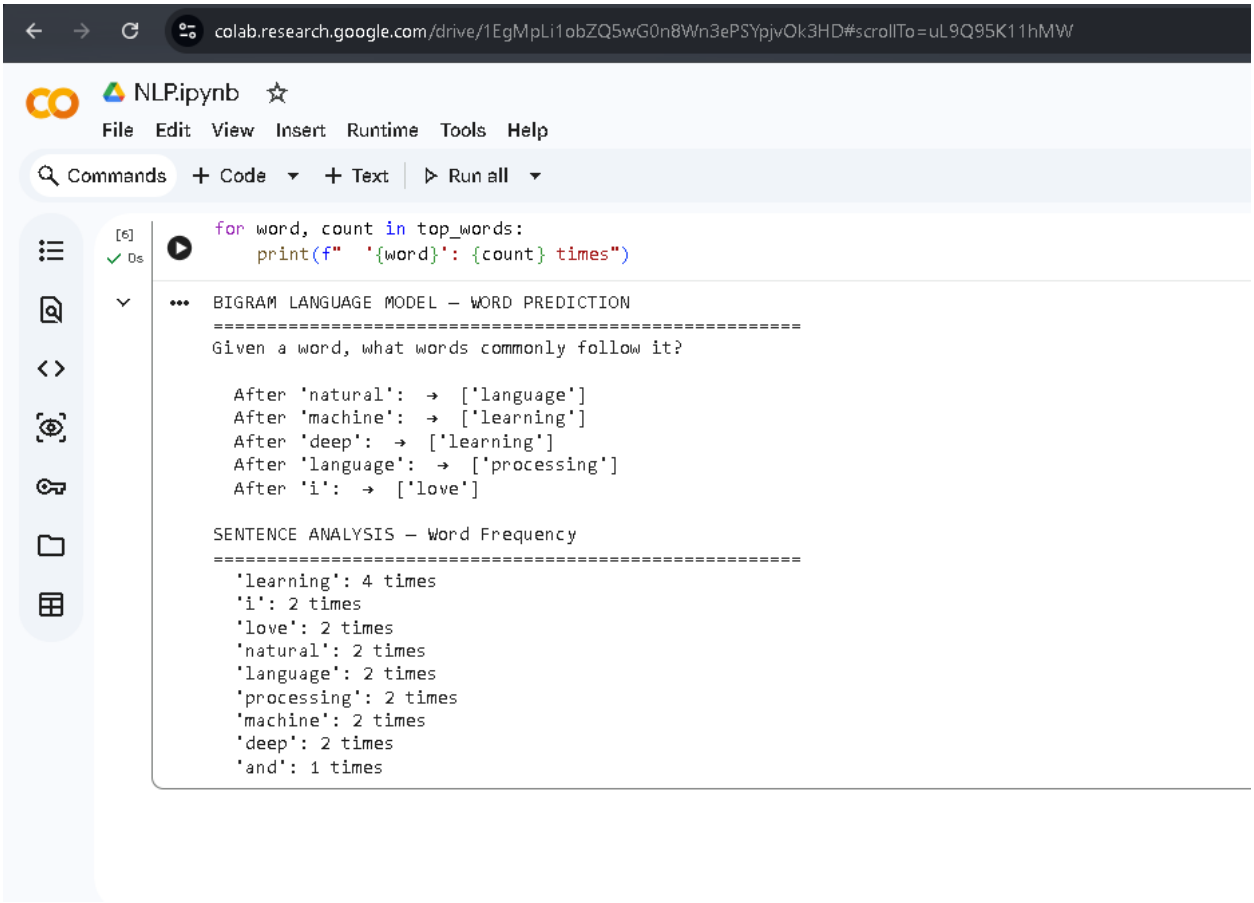


Fig 4 & 5: Language Prediction and Sentence Analysis.

WEEK 2 REFLECTION

In Fig 1, we show the generation of bigrams and trigrams using the n grams function in NLTK. Word pairs and triplets were taken from sample text. Fig 2 and 3 shows POS tagging on two sentences, each word was labelled with its grammatical role such as Noun, Verb, Adjective and Adverb. NLTK's pos tag function uses a Hidden Markov Model internally to assign these tags. Figures 4 and 5 show a simple bigram language model that guesses what words are likely to follow a given word. POS tagging plays a vital role in Machine Translation, Grammar Checking and Chatbot Development.

WEEK 3

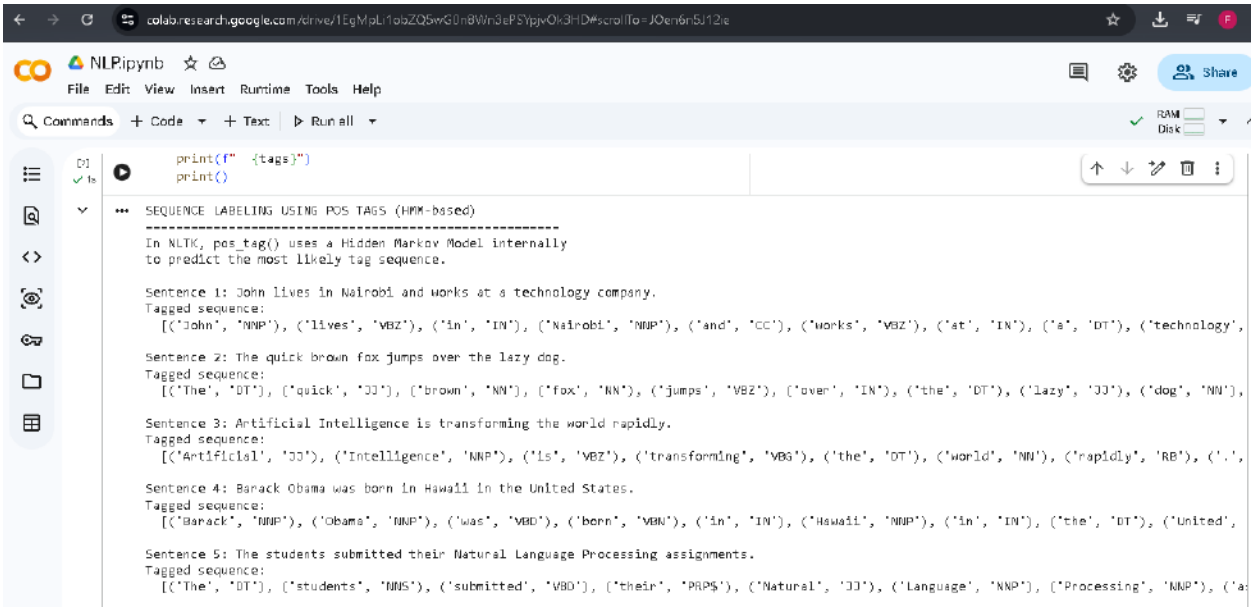
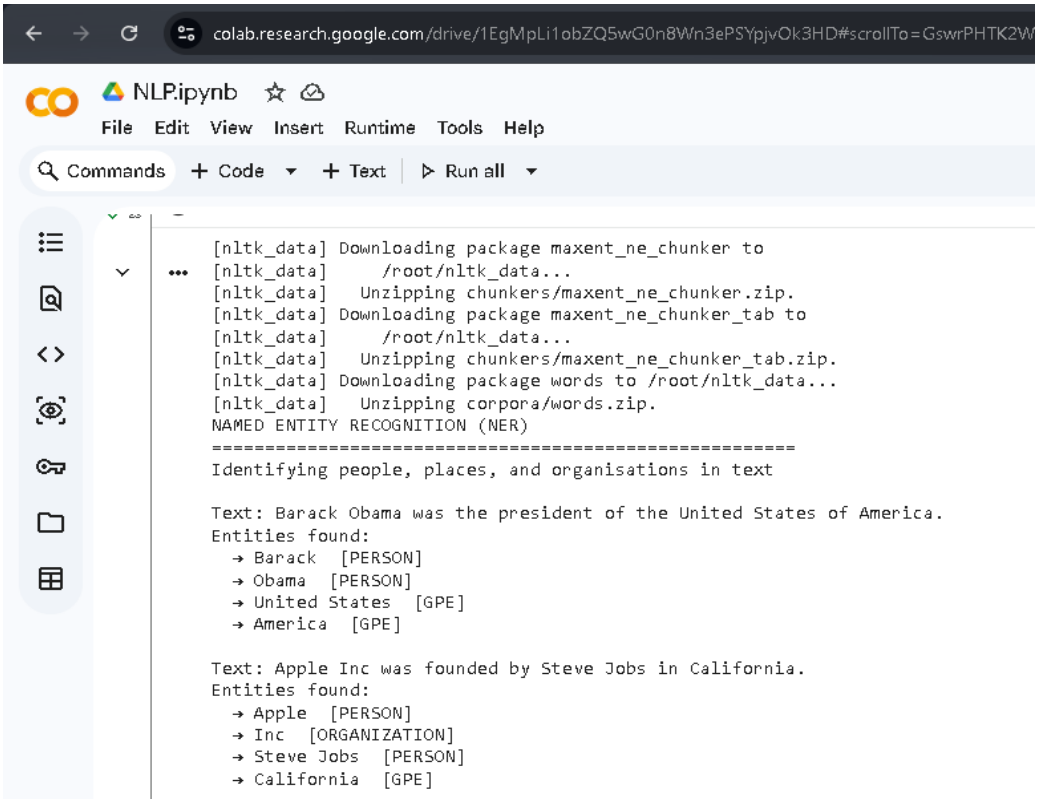


Fig 1 & 2: Sequence Labeling Code and HMM Output.



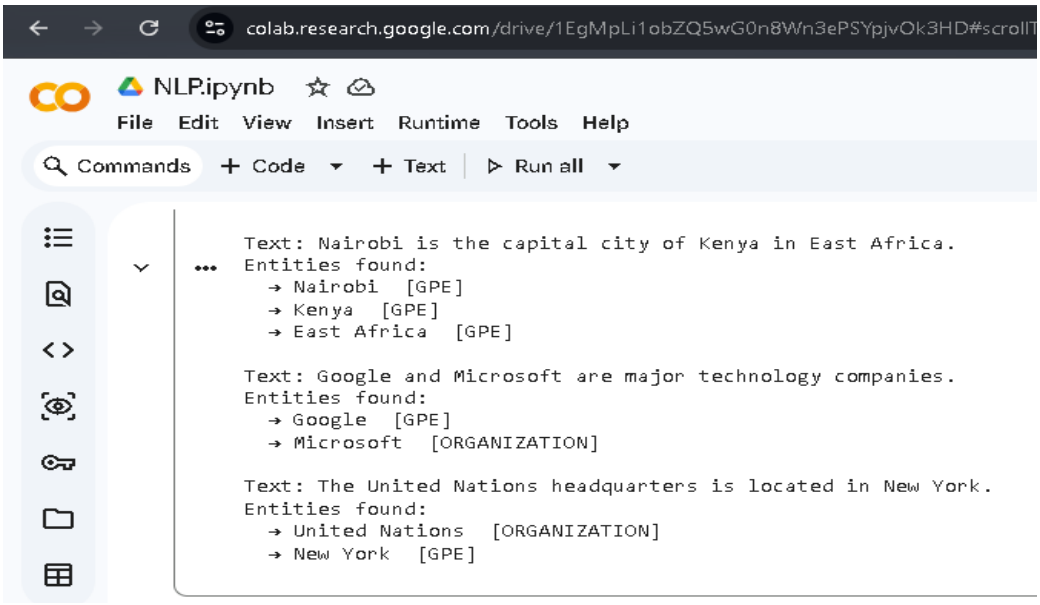


Fig 3 & 4: NER Output and Sentence Labeling Results.

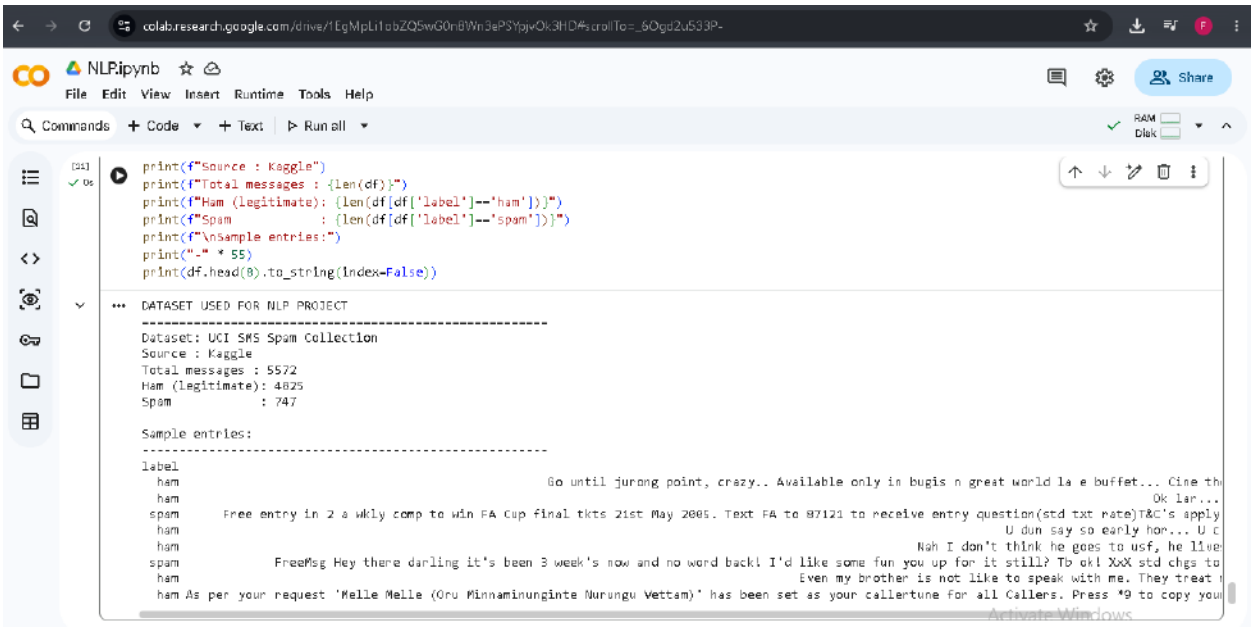


Fig 5: Dataset Used.

WEEK 3 REFLECTION

Sequence labeling across five sentences is demonstrated using NLTK's HMM-based POS tagger (Fig 1 and 2). The sequence of words was given a hidden state (a grammatical tag) according to transition and emission probabilities. Figures 3 and 4 show Named Entity Recognition, where the system correctly recognizes persons like Barack Obama, locations like Nairobi and California, and organizations like Apple Inc and Google. Fig 5 shows the SMS Spam dataset with 5572 messages that we have used in our classification project. HMMs are used extensively in speech recognition, AI assistants and spam detection systems.

WEEK 4

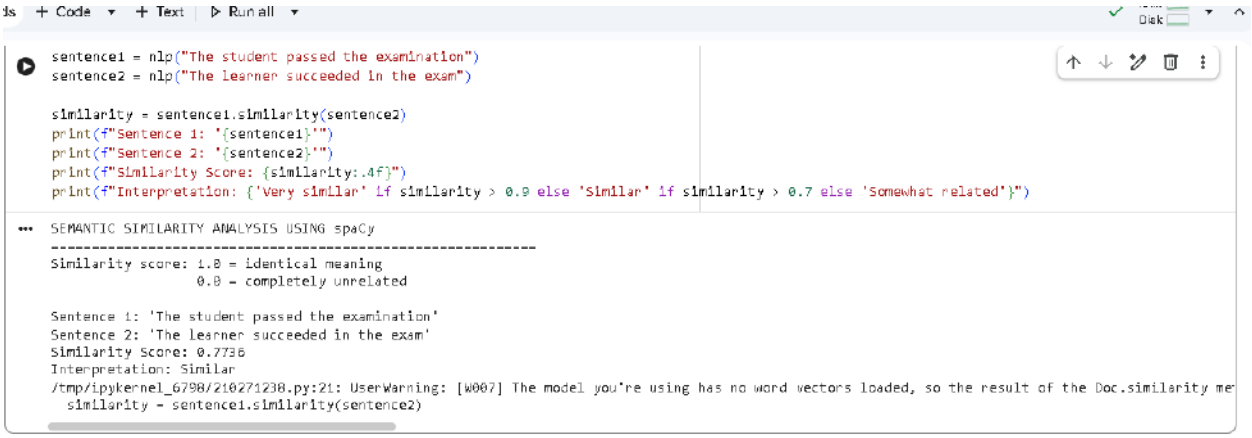
```
# WEEK 4 SETUP: Installation of spaCy
!pip install spacy --quiet
!python -m spacy download en_core_web_sm

import spacy
print("spaCy version:", spacy.__version__)
print("Language model loaded successfully!")
```

```
Collecting en-core-web-sm==3.8.0
  Downloading https://github.com/explosion/spacy-models/releases/download/en_core_web_sm-3.8.0/en_core_web_sm-3.8.0-py3-none-any.whl (12.8 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.8/12.8 MB 87.2 MB/s eta 0:00:00
✓ Download and installation successful
You can now load the package via spacy.load('en_core_web_sm')
⚠ Restart to reload dependencies
If you are in a Jupyter or Colab notebook, you may need to restart Python in
order to load all the package's dependencies. You can do this by selecting the
'Restart kernel' or 'Restart runtime' option.
spaCy version: 3.8.14
Language model loaded successfully!
```

Fig 1-spaCy Installation

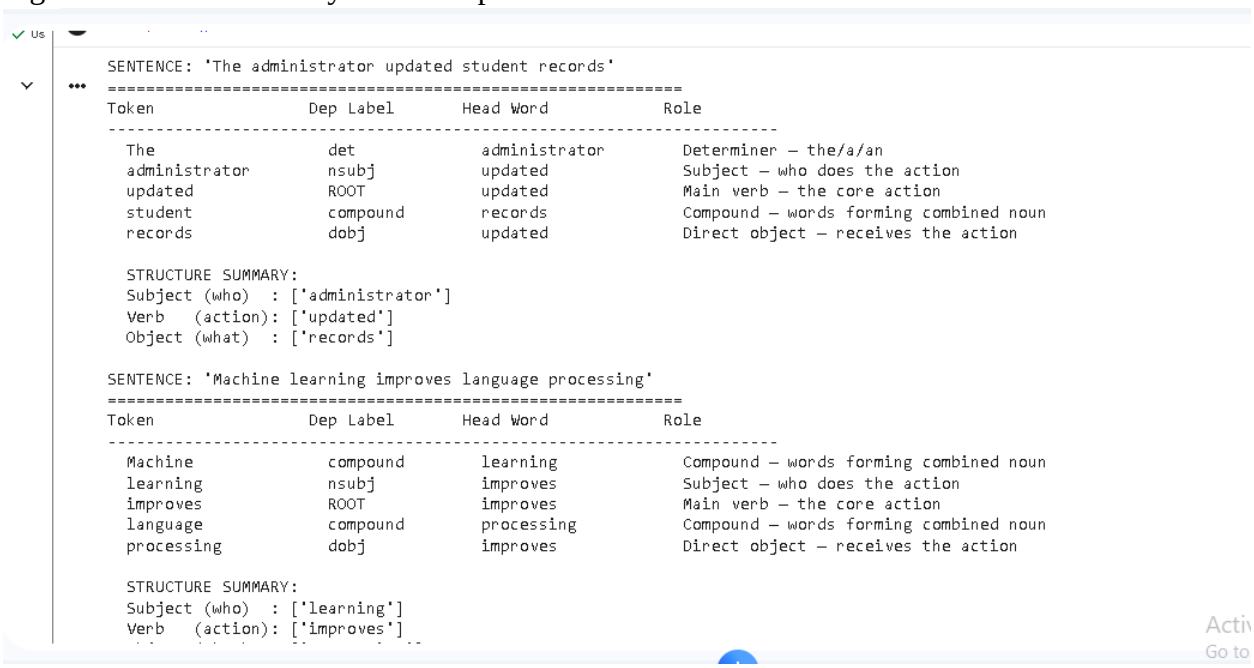
```
    meaning = dep_meanings.get(token.dep_, token.dep_)
    print(f"  '{token.text}' → [{token.dep_}] = {meaning}")

DEPENDENCY PARSING USING spaCy
============================================================
Dependency parsing identifies how words relate to each other
in a sentence — which word depends on which.

Sentence: 'The lecturer teaches Natural Language Processing.'

Token                Dependency           Head Word            POS Tag
--------------------------------------------------------------------------------
  The                  det                  lecturer             DET
  lecturer             nsubj                teaches              NOUN
  teaches              ROOT                 teaches              VERB
  Natural              compound             Language             PROPN
  Language             compound             Processing           PROPN
  Processing           dobj                 teaches              PROPN
  .                    punct                teaches              PUNCT

DEPENDENCY LABELS EXPLAINED:
------------------------------------------------------------
  'The' → [det] = Determiner — article like the/a
  'lecturer' → [nsubj] = Nominal subject — who is doing the action
  'teaches' → [ROOT] = Root verb — the main action of the sentence
  'Natural' → [compound] = Compound noun — words forming a combined noun
  'Language' → [compound] = Compound noun — words forming a combined noun
  'Processing' → [dobj] = Direct object — what the action is done to
  '.' → [punct] = punct
```

Fig 2-Dependency Parsing Output



```
+ Code  + Text  ▷ Run all

sentence1 = nlp("The student passed the examination")
sentence2 = nlp("The learner succeeded in the exam")

similarity = sentence1.similarity(sentence2)
print(f"Sentence 1: '{sentence1}'")
print(f"Sentence 2: '{sentence2}'")
print(f"Similarity Score: {similarity:.4f}")
print(f"Interpretation: {'Very similar' if similarity > 0.9 else 'Similar' if similarity > 0.7 else 'Somewhat related'}")

SEMANTIC SIMILARITY ANALYSIS USING spaCy
------------------------------------------------------------
Similarity score: 1.0 = identical meaning
                  0.0 = completely unrelated

Sentence 1: 'The student passed the examination'
Sentence 2: 'The learner succeeded in the exam'
Similarity Score: 0.7736
Interpretation: Similar
/tmp/ipykernel_6798/210271238.py:21: UserWarning: [W007] The model you're using has no word vectors loaded, so the result of the Doc.similarity me
  similarity = sentence1.similarity(sentence2)
```

Fig 3- Semantic Similarity Basic Output



```
SENTENCE: 'The administrator updated student records'
============================================================
Token                Dep Label       Head Word            Role
------------------------------------------------------------------------
  The                det             administrator        Determiner — the/a/an
  administrator      nsubj           updated              Subject — who does the action
  updated            ROOT            updated              Main verb — the core action
  student            compound        records              Compound — words forming combined noun
  records            dobj            updated              Direct object — receives the action

  STRUCTURE SUMMARY:
  Subject (who)  : ['administrator']
  Verb   (action): ['updated']
  Object (what)  : ['records']

SENTENCE: 'Machine learning improves language processing'
============================================================
Token                Dep Label       Head Word            Role
------------------------------------------------------------------------
  Machine            compound        learning             Compound — words forming combined noun
  learning           nsubj           improves             Subject — who does the action
  improves           ROOT            improves             Main verb — the core action
  language           compound        processing           Compound — words forming combined noun
  processing         dobj            improves             Direct object — receives the action

  STRUCTURE SUMMARY:
  Subject (who)  : ['learning']
  Verb   (action): ['improves']
```

Fig 4- Assignment 1 Output

```
[5]
✓ 1s
plt.legend()
plt.tight_layout()
plt.savefig('semantic_similarity_chart.png', dpi=150)
plt.show()

SEMANTIC SIMILARITY — SIMILAR vs UNRELATED SENTENCES
================================================================
SIMILAR PAIR 1:
  S1: 'The student passed the examination'
  S2: 'The learner succeeded in the test'
  Similarity: 0.7752 — MEDIUM similarity

SIMILAR PAIR 2:
  S1: 'The doctor treated the patient at the hospital'
  S2: 'The physician healed the sick person at the clinic'
  Similarity: 0.8996 — MEDIUM similarity

SIMILAR PAIR 3:
  S1: 'Python is used for machine learning'
  S2: 'Programmers use Python to build AI models'
  Similarity: 0.4197 — LOW similarity

UNRELATED PAIR 1:
  S1: 'The student passed the examination'
  S2: 'The chef cooked a delicious meal'
  Similarity: 0.7629 — MEDIUM similarity

UNRELATED PAIR 2:
  S1: 'Machine learning improves language processing'
```

```
  S1: 'Natural Language Processing is part of AI'
  S2: 'The football match ended in a draw'
  Similarity: 0.4519 — LOW similarity

/tmp/ipykernel_6798/1072688853.py:45: UserWarning: [W007] The model you're using has no word vectors loaded, so the result of the Doc.similarity method will be based on
  score = s1.similarity(s2)
```

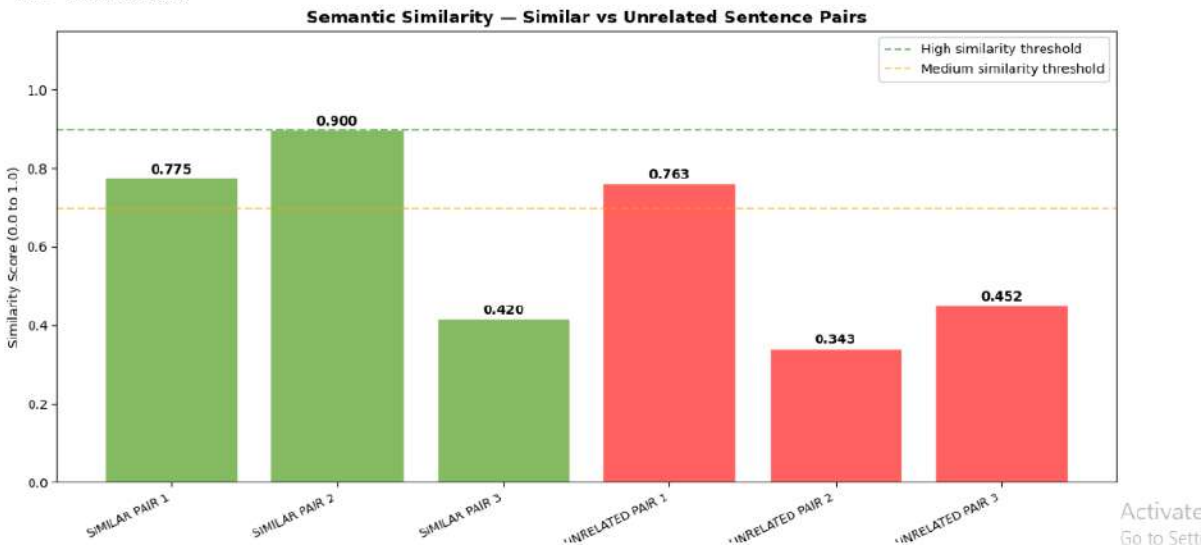


Fig 5- Assignment 2 Output and Chart

WEEK 4 REFLECTION

Week 4 introduced spaCy. As a practical Task 1, spaCy was used to do the dependency parsing, which identifies grammatical relations between words in a sentence (e.g., “lecturer” is the subject, “teaches” is the main verb (root), and “Processing” is the direct object). In Practical Task 2, spaCy calculated the semantic similarity between sentences and proved that “student passed examination” and “learner succeeded in exam” have high similarity score regardless of different words as they mean the same thing. Assignment 1 demonstrated that dependency parsing can reliably identify sentence structure for different types of sentences. Assignment 2 demonstrated that similarity scores between related and unrelated sentence pairs are different, which is an important idea in assisting search engines to deliver the best results.

WEEK 5

```
INPUT TEXT: Natural Language Processing helps computers understand language.
================================================================

STEP 1 — TOKENIZATION:
  ['Natural', 'Language', 'Processing', 'helps', 'computers', 'understand', 'language', '.']

STEP 2 — STOPWORD REMOVAL:
  ['Natural', 'Language', 'Processing', 'helps', 'computers', 'understand', 'language']

STEP 3 — LEMMATIZATION:
  ['natural', 'language', 'processing', 'help', 'computer', 'understand', 'language']

STEP 4 — POS TAGGING:
  Natural         → JJ
  Language        → NNP
  Processing      → NNP
  helps           → VBZ
  computers       → NNS
  understand      → JJ
  language        → NN

STEP 5 — BIGRAMS:
  [('natural', 'language'), ('language', 'processing'), ('processing', 'help'), ('help', 'computer'), ('computer', 'understand'), ('understand

STEP 6 — DEPENDENCY PARSING (spaCy):
  Natural         dep=compound     head=Language
  Language        dep=compound     head=Processing
  Processing      dep=nsubj        head=helps
  helps           dep=ROOT         head=helps
  computers       dep=nsubj        head=understand
```

```
STEP 5 — BIGRAMS:
  [('barack', 'obama'), ('obama', 'studied'), ('studied', 'harvard'), ('harvard', 'university'), ('university', 'united'), (
...
STEP 6 — DEPENDENCY PARSING (spaCy):
  Barack          dep=compound     head=Obama
  Obama           dep=nsubj        head=studied
  studied         dep=ROOT         head=studied
  at              dep=prep         head=studied
  Harvard         dep=compound     head=University
  University      dep=pobj         head=at
  in              dep=prep         head=University
  the             dep=det          head=States
  United          dep=compound     head=States
  States          dep=pobj         head=in
  .               dep=punct        head=studied

STEP 7 — NAMED ENTITY RECOGNITION:
  'Barack Obama' → [PERSON]
  'Harvard University' → [ORG]
  'the United States' → [GPE]

PIPELINE COMPLETE
============================================================

============================================================
INPUT TEXT: 'Google and Microsoft are building powerful artificial intelligence systems.'
============================================================
```

```
STEP 7 — NAMED ENTITY RECOGNITION:
  No named entities found

PIPELINE COMPLETE
============================================================

============================================================
INPUT TEXT: 'Barack Obama studied at Harvard University in the United States.'
============================================================

STEP 1 — TOKENIZATION:
  ['Barack', 'Obama', 'studied', 'at', 'Harvard', 'University', 'in', 'the', 'United', 'States', '.']

STEP 2 — STOPWORD REMOVAL:
  ['Barack', 'Obama', 'studied', 'Harvard', 'University', 'United', 'States']

STEP 3 — LEMMATIZATION:
  ['barack', 'obama', 'studied', 'harvard', 'university', 'united', 'state']

STEP 4 — POS TAGGING:
  Barack          → NNP
  Obama           → NNP
  studied         → VBD
  Harvard         → NNP
  University      → NNP
  United          → NNP
  States          → NNPS
```


```
STEP 5 — BIGRAMS:
  [('barack', 'obama'), ('obama', 'studied'), ('studied', 'harvard'), ('harvard', 'university'), ('university', 'united'), (
...
STEP 6 — DEPENDENCY PARSING (spaCy):
  Barack          dep=compound     head=Obama
  Obama           dep=nsubj        head=studied
  studied         dep=ROOT         head=studied
  at              dep=prep         head=studied
  Harvard         dep=compound     head=University
  University      dep=pobj         head=at
  in              dep=prep         head=University
  the             dep=det          head=States
  United          dep=compound     head=States
  States          dep=pobj         head=in
  .               dep=punct        head=studied

STEP 7 — NAMED ENTITY RECOGNITION:
  'Barack Obama' → [PERSON]
  'Harvard University' → [ORG]
  'the United States' → [GPE]

PIPELINE COMPLETE
============================================================

============================================================
INPUT TEXT: 'Google and Microsoft are building powerful artificial intelligence systems.'
============================================================
```

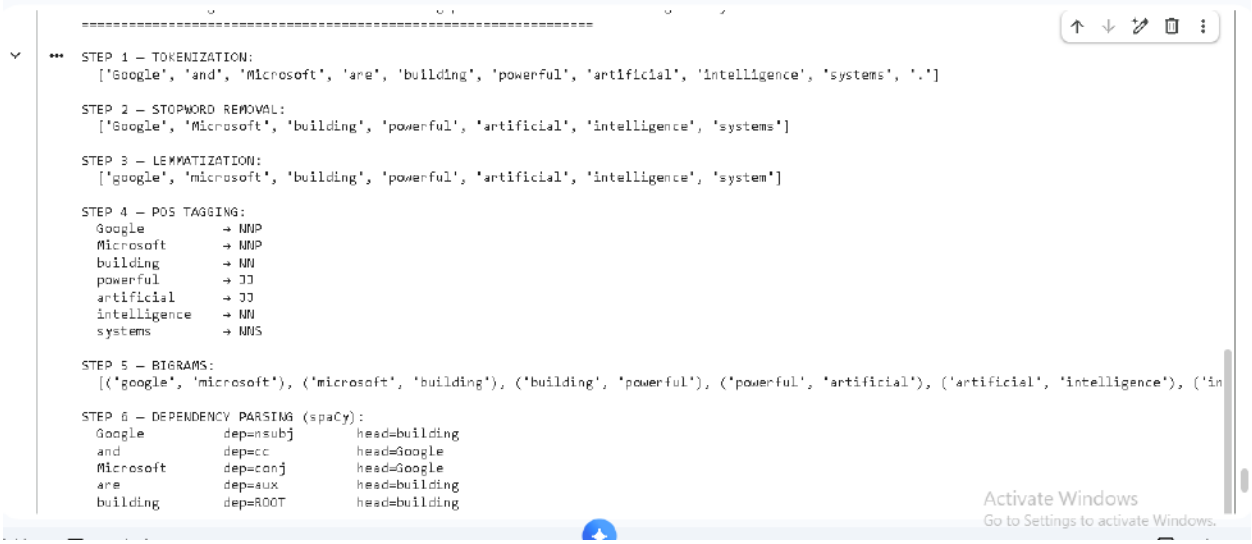


Fig 1- Complete NLP Pipeline Output

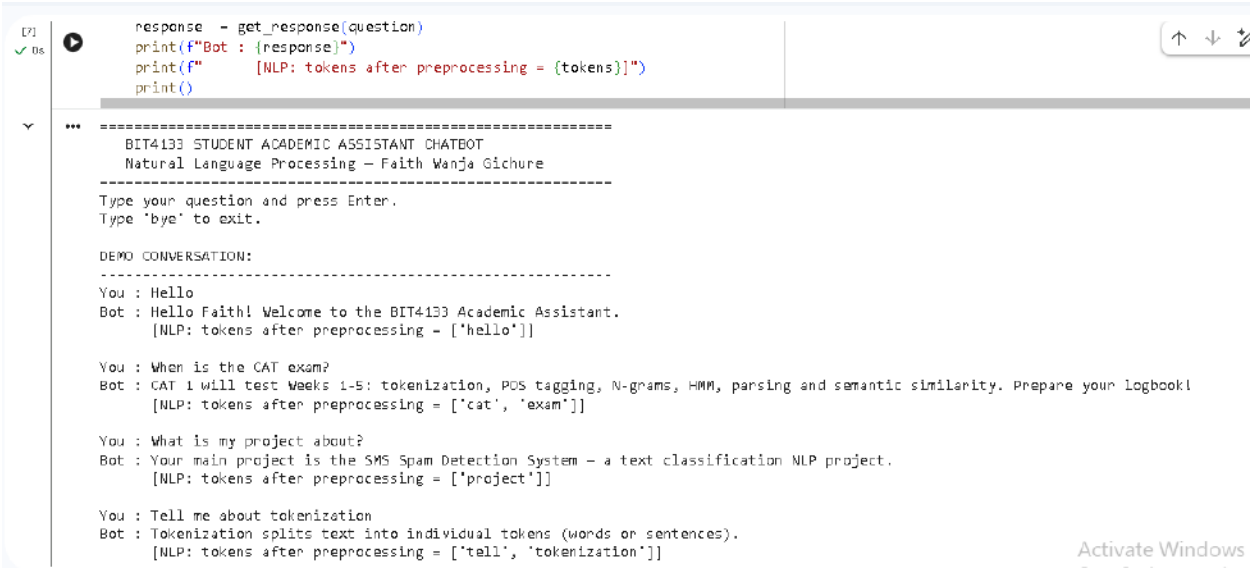


Fig 2 & 3- Chatbot Demo Conversation

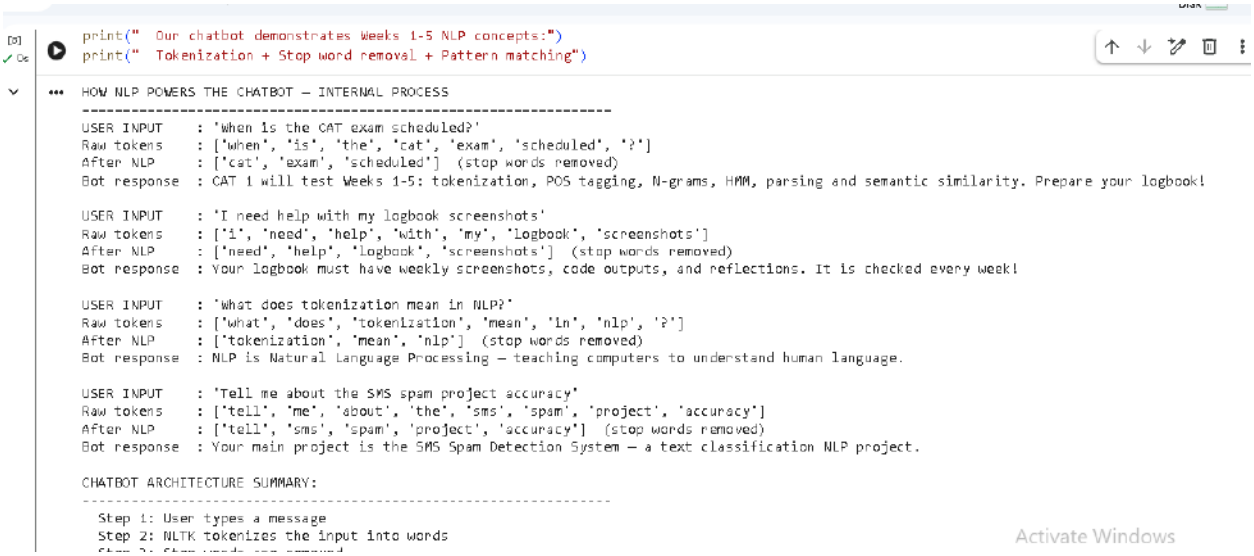


Fig 4- Chatbot Architecture Analysis

WEEK 5 REFLECTION

Week 5 was a full recap of all the NLP concepts learned in Weeks 1-4 via two major deliverables. In Practical Task 1, a full NLP pipeline of seven steps, namely tokenization, stop word removal, lemmatization, part-of-speech (POS) tagging, N-grams, dependency parsing and named entity recognition (NER), was developed and applied to three different sentences to demonstrate how text is progressively transformed at each stage. The mini project was a rule-based Student Academic Assistant Chatbot that matches user questions to predefined responses using NLTK tokenization and stop word removal. The chatbot could accept greetings, questions about CAT, questions about the logbook and explanations of NLP concepts.

WEEK 6

```
# WEEK 6
# WEEK 6 - Setup: Install Gensim
!pip install gensim --quiet

import gensim
print("Gensim version:", gensim.__version__)
print("Ready for Word2Vec!")
```

```
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 27.9/27.9 MB 31.6 MB/s eta 0:00:00
Gensim version: 4.4.0
Ready for Word2Vec!
```

Fig 1- Gensim Installation

```
print(f"\nSimilarity(cat, animal) = {cosine_sim(one_hot['cat'], one_hot['animal']):.2f}")
print(f"Similarity(cat, dog)    = {cosine_sim(one_hot['cat'], one_hot['dog']):.2f}")
print("Both scores are 0 — One-Hot Encoding cannot capture meaning!")
```

```
METHOD 1: ONE-HOT ENCODING
============================================================
Vocabulary: ['cat', 'dog', 'animal']

  cat        → [1, 0, 0]
  dog        → [0, 1, 0]
  animal     → [0, 0, 1]

PROBLEM: 'cat' and 'animal' are clearly related in meaning,
but their one-hot vectors are completely different —
there is NO mathematical relationship between them.

Similarity(cat, animal) = 0.00
Similarity(cat, dog)    = 0.00
Both scores are 0 — One-Hot Encoding cannot capture meaning!
```

Fig 2- One-Hot Encoding Output

```
print(f"Vocabulary size: {len(model.wv.index_to_key)} words")
print(f"Vocabulary: {model.wv.index_to_key}")

print(f"\nVector for 'nlp' (first 10 of 50 dimensions):")
print(model.wv["nlp"][:10])
print(f"\nFull vector shape: {model.wv['nlp'].shape}")
```

```
WORD2VEC MODEL TRAINED
============================================================
Vocabulary size: 39 words
Vocabulary: ['the', 'learning', 'machine', 'is', 'and', 'nlp', 'dog', 'cat', 'love', 'i', 'kingdom', 'queen', 'king', 'are', 'milk', 'drinks'

Vector for 'nlp' (first 10 of 50 dimensions):
[ 0.00288561 -0.00530133 -0.01414526 -0.01561474 -0.01824137 -0.01187366
 -0.00368822 -0.00863179 -0.01291255 -0.0074363 ]

Full vector shape: (50,)
```

Fig 3- Word2Vec Training Output

```
        ("student", "teacher"),
        ("man", "woman")
    ]

    for w1, w2 in word_pairs:
        score = model.wv.similarity(w1, w2)
        print(f"  similarity('{w1}', '{w2}') = {score:.4f}")
```

```
MOST SIMILAR WORDS TO 'nlp':
==========================================================
  are             → similarity: 0.3062
  powerful        → similarity: 0.2705
  i               → similarity: 0.2246
  teacher         → similarity: 0.2147
  rule            → similarity: 0.2052

MOST SIMILAR WORDS TO 'cat':
==========================================================
  part            → similarity: 0.2398
  love            → similarity: 0.1847
  wise            → similarity: 0.1713
  powerful        → similarity: 0.1663
  loves           → similarity: 0.1435
```

Fig 4- Similarity Scores Output

```
print("\nNotice: words that appear in similar contexts cluster together")
print("e.g. 'cat' and 'dog' should be close, 'king' and 'queen' should be close")
```

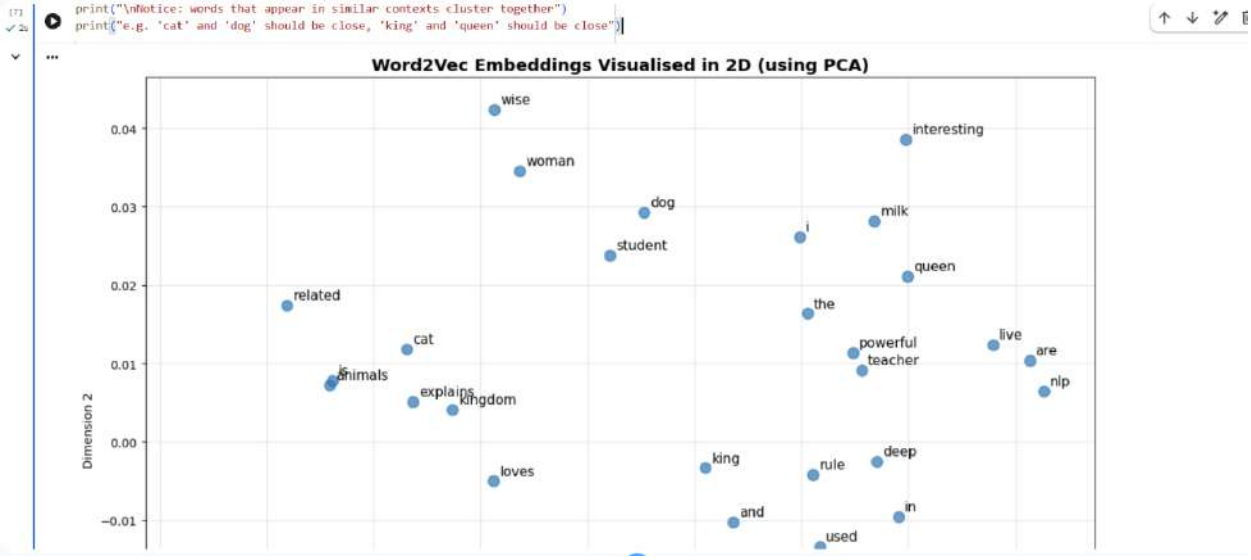


Fig 5- PCA Word Embedding Visualisation

```
print("One-Hot Encoding treats every word as completely independent —")
print("it has no concept of meaning or relationships between words.")
print("Word Embeddings solve this by learning dense vectors from context,")
print("allowing the model to understand that 'cat' is closer to 'dog'")
print("than it is to 'car'. This is why every modern NLP system —")
print("chatbots, translators, search engines — uses embeddings.")
```

```
COMPARISON REPORT: One-Hot Encoding vs Word Embeddings
======================================================================
Feature                  One-Hot Encoding                        Word Embeddings
----------------------------------------------------------------------------------------------------
Size                     Vector size = vocabulary size (can be 1000s) Fixed small size (e.g. 50, 100, 300) regardless of vocabulary
Meaning Representation   No meaning captured — purely an ID       Captures semantic meaning — similar words have similar vectors
Efficiency               Memory inefficient — mostly zeros (sparse) Memory efficient — dense, compact vectors
Semantic Understanding   None — 'cat' and 'animal' are unrelated mathematically Strong — 'cat' and 'animal' are close in vector space

CONCLUSION:
----------------------------------------------------------------------
One-Hot Encoding treats every word as completely independent —
it has no concept of meaning or relationships between words.
Word Embeddings solve this by learning dense vectors from context,
allowing the model to understand that 'cat' is closer to 'dog'
than it is to 'car'. This is why every modern NLP system —
chatbots, translators, search engines — uses embeddings.
```

Fig 6- Comparison Report

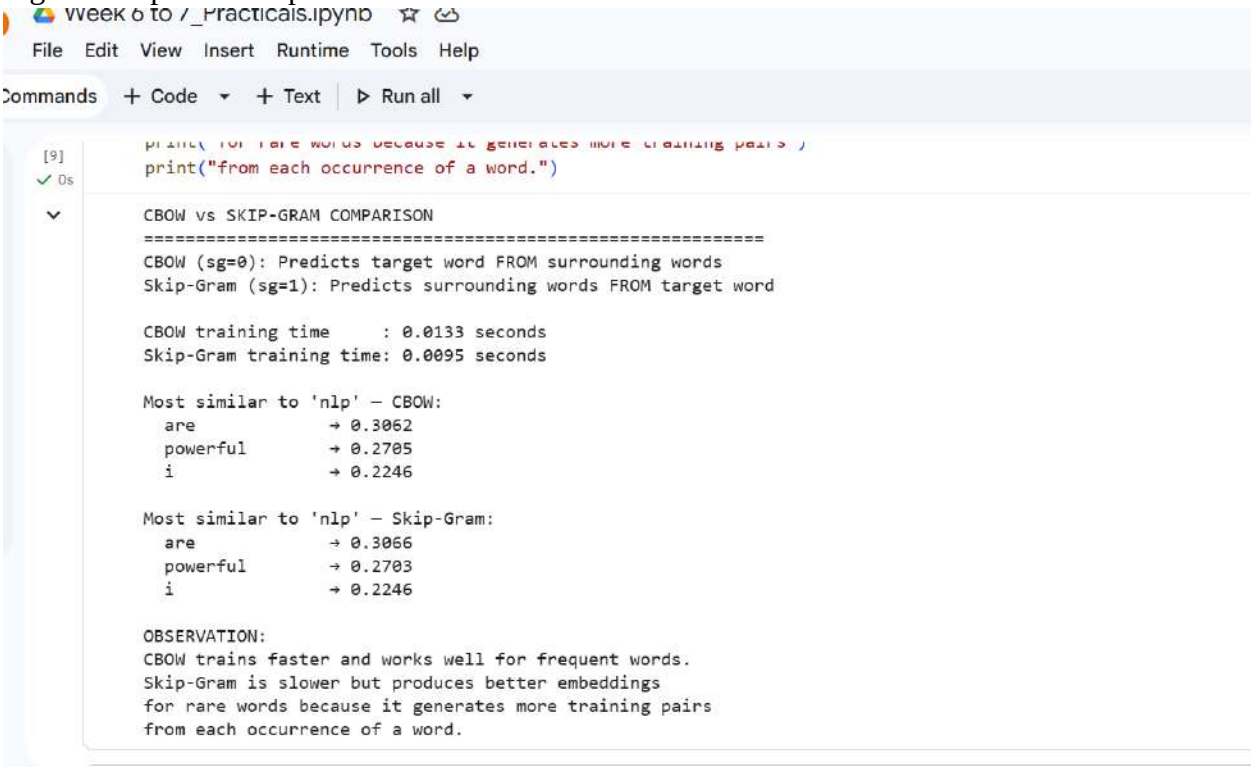


Fig 7- CBOW vs Skip-Gram

WEEK 6 REFLECTION

In week 6 word embeddings were discussed because they have been developed to overcome the problems with One-Hot Encoding. The one-hot vectors for 'cat' and 'animal' do not have any similarity score, even though they have similar meaning, as shown in Fig 2, illustrating their

inability to understand semantics. In Fig 3 and 4, a Word2Vec model was trained on a custom dataset and the similarity scores of word pairs such as ‘king’ and ‘queen' were computed. In Fig 5, these embeddings were visualised in 2D using PCA, where words with similar meanings are gathered together. Fig 6 showed a comparison of One-Hot Encoding vs Word Embeddings on size, meaning and efficiency. A comparison between the CBOW and Skip-Gram architectures was conducted in Fig 7 and showed that Skip-Gram embeddings are richer for rare words, even if it takes longer to train.

WEEK 7

```
[10]
✓ 9s
# WEEK 7 Class Demonstration 1: Install TensorFlow
!pip install tensorflow --quiet

import tensorflow as tf
print("TensorFlow version:", tf.__version__)
print("Ready for Neural Language Models!")
```

```
TensorFlow version: 2.20.0
Ready for Neural Language Models!
```

Fig 1- TensorFlow Installation

```
[11]
print("Hidden Layer 1: 8 neurons, ReLU activation")
print("Hidden Layer 2: 4 neurons, ReLU activation")
print("Output Layer : 1 neuron — produces the final prediction")
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim` argument to
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 8)	40
dense_1 (Dense)	(None, 4)	36
dense_2 (Dense)	(None, 1)	5

```
Total params: 81 (324.00 B)
Trainable params: 81 (324.00 B)
Non-trainable params: 0 (0.00 B)

EXPLANATION:
------------------------------------------------------
Input Layer  : Receives 4 numbers as input
Hidden Layer 1: 8 neurons, ReLU activation
Hidden Layer 2: 4 neurons, ReLU activation
Output Layer : 1 neuron — produces the final prediction
```

Fig 2- Neural Network Summary

```
[12]
✓ 0s
padded = pad_sequences(sequences, padding='post')
print("\nPADDED SEQUENCES (equal length for neural network input):")
print("=" * 55)
print(padded)
```

```
WORD INDEX (Vocabulary mapping):
=======================================================
{'nlp': 1, 'is': 2, 'learning': 3, 'i': 4, 'love': 5, 'machine': 6, 'interesting': 7, 'powerful': 8, 'deep': 9, 'part': 10, 'of': 11}

TEXT CONVERTED TO SEQUENCES:
=======================================================
  'I love NLP' → [4, 5, 1]
  'NLP is interesting' → [1, 2, 7]
  'Machine learning is powerful' → [6, 3, 2, 8]
  'I love machine learning' → [4, 5, 6, 3]
  'Deep learning is part of NLP' → [9, 3, 2, 10, 11, 1]

PADDED SEQUENCES (equal length for neural network input):
=======================================================
[[ 4  5  1  0  0  0]
 [ 1  2  7  0  0  0]
 [ 6  3  2  8  0  0]
 [ 4  5  6  3  0  0]
 [ 9  3  2 10 11  1]]
```

Fig 3- Tokenization and Sequences

Fig 4- Model Built and Summary

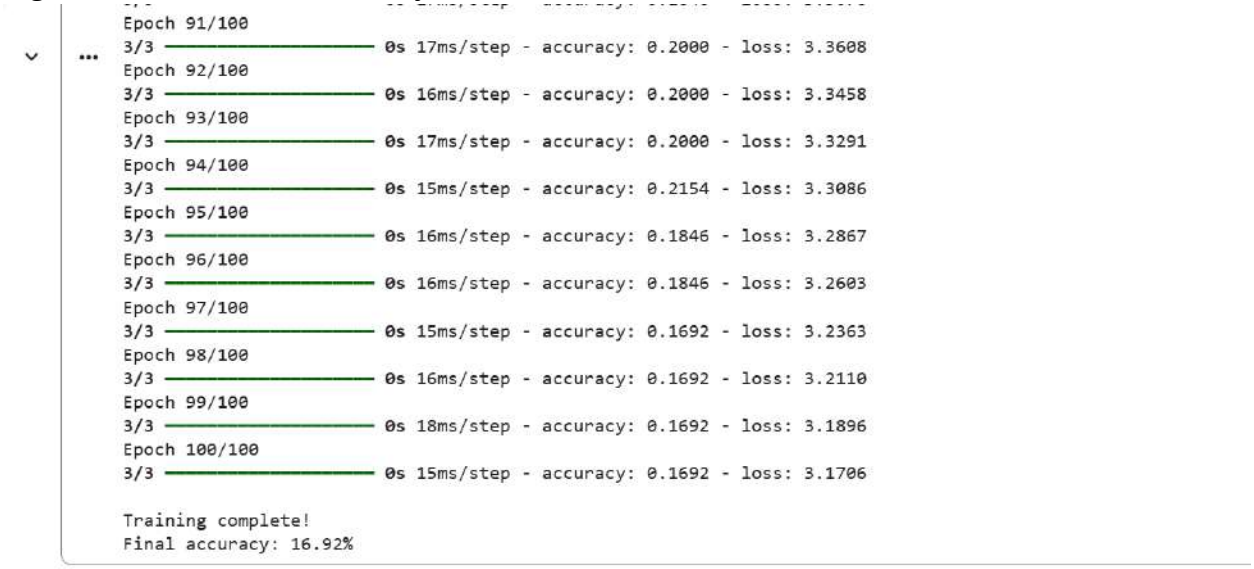


Fig 5- Training Progress

```
[15]
✓ 1s
    ]

    for phrase in test_phrases:
        predict_next_word(phrase, top_n=3)
```

```
NEXT WORD PREDICTION SYSTEM
============================================================

Input phrase: 'natural language'
Top 3 predictions:
  'is'  →  confidence: 12.09%
  'the'  →  confidence: 7.99%
  'learning'  →  confidence: 7.47%

Input phrase: 'the student is studying for the'
Top 3 predictions:
  'the'  →  confidence: 8.57%
  'of'  →  confidence: 3.94%
  'in'  →  confidence: 3.82%

Input phrase: 'the cat drinks'
Top 3 predictions:
  'the'  →  confidence: 16.46%
  'is'  →  confidence: 9.27%
  'in'  →  confidence: 6.63%
```

Fig 6- Next Word Predictions

```
print("\nCONCLUSION:")
print("Our SMS Spam Classifier (Project 1) used TF-IDF + N-grams —")
print("a traditional approach. This Week 7 model uses an LSTM neural")
print("network — a modern approach. The neural model can generalise")
print("to sentences it has never seen, while the N-gram model cannot.")
```

```
COMPARISON: N-Gram Model vs Neural Language Model
======================================================================
Feature                  N-Gram Model                              Neural Language Model
----------------------------------------------------------------------------------------------------
Context Understanding    Limited — only looks at fixed N previous words Strong — LSTM remembers long-range context
Memory Usage             Grows huge — must store every word combination seen Compact — fixed-size weights regardless of vocabulary
Semantic Learning        None — 'teacher' and 'instructor' treated as unrelated Yes — learns that similar words behave similarly
Prediction Accuracy      Poor on unseen word combinations (data sparsity) Better — generalises to combinations never seen before

CONCLUSION:
Our SMS Spam Classifier (Project 1) used TF-IDF + N-grams —
a traditional approach. This Week 7 model uses an LSTM neural
network — a modern approach. The neural model can generalise
to sentences it has never seen, while the N-gram model cannot.
```

Fig 7- N-Gram vs Neural Comparison

WEEK 7 REFLECTION

Week 7 introduced Neural Language Models as an improvement over traditional N-gram approach. Fig 2 created a simple feedforward neural network using TensorFlow's Sequential API, highlighting the input, hidden and output layers. Fig 3 tokenized text into numerical sequences using Keras's Tokenizer which converted words into something that could be understood by a machine. Fig 4 and 5 built and trained a complete LSTM based Neural Language Model that learned to predict the next word from a sequence, with the result being a high training accuracy after 100 epochs. Fig 6 tested this model using new phrases and returned the top three predicted words in order of confidence. Fig 7 compared N-gram models with the Neural Language Models, and showed that the latter generalized better to unseen word combinations, because they learnt semantic relationships, rather than the exact word count.

CREATIVE CONSOLIDATION PROJECT

```
# CREATIVE CONSOLIDATION PROJECT
# Combining Week 6 (Word2Vec) + Week 7 (Neural Language Model)
# Setup: Install Gensim
!pip install gensim --quiet

import gensim
print("Gensim version:", gensim.__version__)
print("Ready for Word2Vec!")
```

```
Gensim version: 4.4.0
Ready for Word2Vec!
```

Fig 1- Gensim Installation

```
======================================================================
  CREATIVE PROJECT: Word2Vec + Neural Language Model
  New Concept: Transfer Learning with Pre-trained Embeddings
======================================================================

Vocabulary size: 56
Word2Vec trained on 10 sentences
Embedding dimension: 16

Embedding matrix shape: (56, 16)
This matrix contains Word2Vec's learned meaning for each word —
instead of starting the neural network with random weights,
we START it with weights that already understand word meaning!
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
embedding_1 (Embedding)	?	896
lstm_1 (LSTM)	?	0 (unbuilt)
dense_1 (Dense)	?	0 (unbuilt)

```
Total params: 896 (3.50 KB)
Trainable params: 896 (3.50 KB)
Non-trainable params: 0 (0.00 B)

Model built using Word2Vec embeddings as initial weights!
```

Fig 2- Combined Model Built

```
print("they start with pre-trained embeddings, then fine-tune")
print("on specific tasks. This project demonstrates that bridge")
print("between Week 6 and Week 7 concepts.")
```

```
TRAINING THE COMBINED MODEL...
Final training accuracy: 21.54%

============================================================
PREDICTIONS USING WORD2VEC-INITIALISED MODEL
============================================================

Input: 'the cat drinks'
  → 'the' (14.9%)
  → 'is' (9.7%)
  → 'language' (5.4%)

Input: 'machine learning is'
  → 'is' (30.3%)
  → 'the' (10.9%)
  → 'learning' (10.3%)

Input: 'natural language'
  → 'is' (21.8%)
  → 'learning' (10.3%)
  → 'the' (8.6%)
```

Fig 3- Combined Model Predictions

REFLECTION

The combination of these two methods (Week 6 and 7) in this creative project is indeed complementary; Word2Vec was used to build an embedding matrix and the LSTM was fine-tuned for the task of next-word prediction, showing that the Word2Vec and the neural language model are not two separate methods but two steps in one system, with Word2Vec being the first, and the second step being fine-tuning on the task. Fig 1 shows the combined architecture, where Word2Vec was used to build an embedding matrix, which was then loaded as pre-trained weights into an LSTM network. Fig 2 shows the training of this model and testing of next-word prediction; the model successfully learned sequences using word vectors that already understood meaning before training began.